

A Benchmark for Sinhala-Script Language Identification: Disambiguating Sinhala, Pali, and Sanskrit in Closed and Open-World Contexts

Anonymous ACL submission

Abstract

Multilingual language identification (LangID) models often treat scripts primarily used in a single country as exclusive proxies for its national language (Caswell et al., 2020). For the Sinhala script (U+0D80–U+0DFF), this overlooks canonical Pali and classical Sanskrit, written in this script for over two millennia (Adikaram, 1946). Consequently, leading foundation models (fastText, GlotLID, NLLB) show **0.0% zero-shot recall** on Sinhala-script Pali and Sanskrit, mislabeling all text as modern Sinhala and degrading downstream OCR and translation. We present the first dedicated LangID benchmark for these languages in their shared native script across closed- and open-world settings. On a document-stratified corpus of 74,318 sentences, baselines achieve > 0.98 Macro-F1 on full sentences. However, single-word tests reveal that character n -gram models substantially outperform neural sequence models (0.858 vs. 0.610 F1) by capturing inflectional affixes. In open-world benchmarks across FLORES+, CommonLID, and WiLI-2018, naive fine-tuning induces catastrophic forgetting in XLM-RoBERTa (0.117 Macro-F1), whereas continual rehearsal with multilingual data (Aya) (Singh et al., 2024) restores accuracy (0.874 for XLM-R; > 0.994 for GlotLID and NLLB).

1 Introduction

Language Identification (LangID) is a fundamental first step in multilingual text processing. It filters web crawls, routes incoming text to translation systems, and categorizes digital library collections (Jauhiainen et al., 2019; Lui and Baldwin, 2012; Caswell et al., 2020). While existing models successfully handle script-sharing across languages that use Latin or Cyrillic, they often struggle when a script is primarily associated with a single country. In such cases, models tend to treat the script as an exclusive proxy for that country’s dominant

language (Kargaran et al., 2023; Costa-Jussà et al., 2022).

This assumption fails for the Sinhala script (U+0D80–U+0DFF). While the script is primarily used for **Sinhala**, an Indo-Aryan language spoken by approximately 17 million people in Sri Lanka, it has also been used for over two millennia to write canonical **Pali**—the liturgical language of Theravada Buddhism—and classical **Sanskrit** (Walshe, 1983). In fact, the Theravada Buddhist canon (*Tipitaka*) was first committed to writing in the Sinhala script during the first century BCE at the Aluvihara temple (Adikaram, 1946). Consequently, thousands of historical palm-leaf manuscripts (*ola*) and modern printed editions are written entirely in the Sinhala script.

Because current LangID models—such as fastText LID-176 (Joulin et al., 2017), GlotLID v3 (Kargaran et al., 2023), NLLB LID-218 (Costa-Jussà et al., 2022), OpenLID-v2 (Burchell et al., 2023), ConLID (Foroutan et al., 2026), and XLM-RoBERTa (Conneau et al., 2020)—map the Sinhala Unicode block exclusively to modern Sinhala, they show **0.0% recall** on Sinhala-script Pali and Sanskrit (Table 3). Every Pali chant, Buddhist verse, and Sanskrit inscription written in Sinhala script is misclassified as modern Sinhala, polluting multilingual corpora and digital archives.

In this paper, we present the first dedicated LangID benchmark and empirical study for Sinhala, Pali, and Sanskrit in the Sinhala script across closed-world and open-world settings. Specifically, we investigate: (1) how standard baseline architectures distinguish these three languages across decreasing input lengths from full sentences down to isolated words; (2) what architectural barriers prevent classical classifiers from scaling to hundreds of global languages; and (3) how pretrained foundation models can be adapted to resolve local script ambiguities without suffering catastrophic forgetting on global languages. To answer these

Table 1: Target corpus statistics ($N = 74,318$), partitioned with document-level grouping to prevent data leakage.

Language Class	Train	Val	Test	Total
Sinhala (Sinh-Sinh)	26,675	2,895	2,693	32,263
Pali (Pali-Sinh)	23,490	2,800	3,027	29,317
Sanskrit (San-Sinh)	10,120	1,291	1,327	12,738
Total	60,285	6,986	7,047	74,318

questions, we construct a document-stratified corpus of 74,318 sentences, evaluate seven baseline models across four input lengths, analyse the scalability limits of classical classifiers, and benchmark six foundation models on three 11-language hybrid test suites (FLORES+, CommonLID, and WiLI-2018).

2 Dataset and Experimental Setup

2.1 Target Corpus Construction

A common issue when evaluating text classifiers on historical or religious literature is data leakage from random sentence splits. In canonical Buddhist texts and historical chronicles, formulaic greetings, recurring epithets, and standardized verses appear across multiple sentences. If sentences from the same text are shuffled randomly into train and test sets, models can achieve high scores simply by memorizing specific phrases rather than learning genuine grammatical distinctions. To prevent this, we split our data strictly by document (81.1% Train, 9.4% Val, 9.5% Test; Table 1), spanning: (1) **Sinhala** (Sinh-Sinh, 32,263 sentences) drawn from SiPaKosa and parallel corpora; (2) **Pali** (Pali-Sinh, 29,317 sentences) from canonical *Tipitaka* treatises, *Atthakatha* commentaries, and SiDiaC-v2.0 (Jayatilleke and de Silva, 2025); and (3) **Sanskrit** (San-Sinh, 12,738 sentences) from the historical 19th-century *SansinNT* (7,953 sentences), Digital Corpus of Sanskrit (DCS; 4,684 sentences) (Hellwig, 2010) transliterated via *Aksharamukha* (Rajan, 2023), and 101 protective stanzas from SiDiaC-v2.0.

2.2 Multilingual Evaluation Suites

In practical settings, Sinhala-script text appears alongside regional neighbors and international languages. We adopt the standardized **BCP-47 taxonomy** (Phillips and Davis, 2009): (1) **Target Triad**: Sinh-Sinh, Pali-Sinh, San-Sinh; (2) **Regional Neighbors**: San-Deva (Devanagari

Table 2: Macro-F1 of baseline architectures trained from scratch across varying input lengths on held-out target test data ($N = 7,047$).

Architecture	Full Sent.	5-Word	3-Word	1-Word
Multinomial NB	0.9947	0.9782	0.9520	0.8584
fastText (Scratch)	0.9958	0.9768	0.9416	0.8027
Char-CNN	0.9950	0.9749	0.9388	0.7654
Char-BiGRU	0.9948	0.9710	0.9255	0.6916
Linear SVM	0.9960	0.9801	0.9532	0.6099
Logistic Regression	0.9955	0.9780	0.9482	0.6054
XGBoost	0.9842	0.9554	0.9102	0.5891

Sanskrit), Hin-Deva (Hindi), Ben-Beng (Bengali), and Tam-Tam1 (Tamil); and (3) **Global Benchmarks**: Eng-Latn (English), Fre-Latn (French), Ger-Latn (German), and Ara-Arab (Arabic). We evaluate across three hybrid evaluation suites by mixing our held-out 7,047 target sentences into the official test splits of **FLORES+** (Goyal et al., 2022), **CommonLID** (Suarez et al., 2026), and **WiLI-2018** (Thoma, 2018).

3 Closed-World Evaluation and Scalability

3.1 Model Comparison across Input Lengths

We trained seven standard architectures from scratch on our target training set (Table 1): Multinomial Naive Bayes (MNB), Linear SVM, Logistic Regression (LogReg), XGBoost, Char-CNN, Char-BiGRU, and fastText (details in Appendix C).

Sentence-Level Classification. On complete sentences, every model scores above 0.984 Macro-F1 (Table 2), led by Linear SVM at 0.9960. This indicates that when full sentence context is available, separating these three languages is straightforward for standard classifiers.

Performance on Short Fragments. In web queries, catalog lookups, or fragmented OCR lines from damaged palm leaves, input text often consists of only a few words. To test resilience under reduced context, we truncated each test sentence down to its first 5 words, 3 words, and single words. As shown in Table 2, performance diverges noticeably on single words: Linear SVM plummets from 0.9960 to 0.6099 (a 38.6-point drop), Char-BiGRU falls to 0.6916, and XGBoost dips to 0.5891. In contrast, **Multinomial Naive Bayes maintains an F1 of 0.8584**, and **fastText achieves 0.8027**.

This divergence stems from morphological processing. Deep sequence models and margin classifiers rely on broad contextual patterns. In isolated words, this context vanishes. However, Sinhala,

Pali, and Sanskrit are highly synthetic languages with diagnostic inflectional affixes: Pali preserves case endings (genitive *-assa*, instrumental *-ēhi*) and verbal markers (*-ti*); Sanskrit features heavy consonant clusters, visarga, and dual/ablative markers (*-āt*); while Sinhala exhibits modern postpositions and colloquial verbal suffixes (*-vē*, *-ēya*, *-da*). Because Multinomial Naive Bayes and fastText model character n -gram frequencies directly, they detect these sub-word markers even in isolated words without sentence syntax.

3.2 Scalability Limits of Classical Classifiers

Given that simple models like Naive Bayes and Linear SVM achieve ≥ 0.994 Macro-F1 on full sentences, why not simply train them across all 176+ world languages? We identify four fundamental bottlenecks: (1) **Vocabulary Explosion** $\mathcal{O}(|V|)$: While character 1–5-grams require $\approx 4.5 \times 10^4$ features for Sinhala script, scaling across $K \geq 176$ languages over 30 distinct Unicode writing systems expands the vocabulary to $|V_{\text{global}}| > 2.5 \times 10^7$ dimensions, exhausting RAM in kernel computation. (2) **Linear Overhead** $\mathcal{O}(K \cdot D)$: Flat One-vs-Rest or Softmax scoring requires $\mathcal{O}(K \cdot D)$ operations per prediction, whereas fastText organizes classes into a Hierarchical Softmax Huffman tree (Joulin et al., 2017) running in $\mathcal{O}(\log_2 K)$. (3) **Lack of Dense Subword Sharing**: Orthogonal n -gram dimensions cannot transfer shared morphological roots across related Indo-Aryan and Dravidian tongues, unlike continuous embeddings. (4) **Data Ingestion Barrier**: Crawling and balancing web data for 200+ languages requires petabytes of text; adapting existing foundation models is far more compute- and data-efficient. (Detailed formulations are in Appendix D).

4 Open-World Multilingual Evaluation

4.1 Zero-Shot Performance and Fine-Tuning

We evaluated six prominent foundation models on our 11-language hybrid test suites (Table 3): fastText LID-176, OpenLID-v2, NLLB LID-218, GlotLID v3, ConLID, and XLM-RoBERTa Base.

Zero-Shot Performance. Without adaptation, all six models fail to identify Sinhala-script Pali and Sanskrit, producing **0.0000 F1** (Table 3, Left). Because their pretraining label sets map the Sinhala Unicode block exclusively to modern Sinhala, both Pali-Sinh and San-Sinh are misclassified as Sinhala, dragging modern Sinhala’s F1

down to ≈ 0.791 . Meanwhile, Devanagari Sanskrit (San-Deva) remains recognized reliably by most models (≥ 0.959), though it drops in OpenLID-v2 (0.0350 F1).

Catastrophic Forgetting in Target-Only Tuning. When fine-tuned strictly on the three target languages (Table 3, Middle), models learn the target triad with high accuracy (> 0.98 target F1), but background retention diverges sharply: (1) **Total Collapse in XLM-R**: Standard cross-entropy updates distort shared self-attention layers; without replay signals, the model predicts only target classes, wiping all non-target languages to **0.0000 F1** and collapsing overall Macro-F1 to **0.1170** on FLORES+, **0.1074** on CommonLID, and **0.1234** on WiLI-2018. (2) **Stability in Dedicated Models**: GlotLID v3, NLLB LID-218, and ConLID preserved their pre-existing language label heads and embedding spaces during continued supervised training, retaining > 0.92 Macro-F1. (3) **Localized Degradation**: Even with continued training, localized drift occurs: fastText English F1 drops to 0.7074 on FLORES+, and ConLID Arabic drops to 0.6181.

4.2 Mitigating Catastrophic Forgetting with Rehearsal

To prevent catastrophic forgetting, we use continual rehearsal by mixing background language examples sampled from the multilingual **Aya dataset** (Singh et al., 2024), optimizing a balanced replay objective $\mathcal{L} = \alpha \mathcal{L}_{\text{target}} + (1 - \alpha) \mathcal{L}_{\text{rehearsal}}$ with $\alpha = 0.5$. Rehearsal delivers decisive improvements (Table 3, Right): (1) **Restoring XLM-RoBERTa**: Reverses catastrophic forgetting, raising Macro-F1 by +75.7 **points** to 0.8740 on FLORES+ (English rebounds to 0.9646, Hindi to 0.9975, German to 0.9995), 0.8381 on CommonLID, and 0.8247 on WiLI-2018. (2) **High Open-World Accuracy**: GlotLID v3 and NLLB LID-218 lead overall, achieving 0.9954 and 0.9948 Macro-F1 on FLORES+, with consistent > 0.99 accuracy across both target classes and global background languages. (3) **Resolving Localized Drops**: ConLID Arabic F1 recovers from 0.6181 to 0.9877, while fastText English F1 improves to 0.8178 on FLORES+ and 0.9565 on WiLI-2018.

4.3 Inference Speed Comparison

While adapted XLM-R and fastText-based models achieve strong F1, their operational profiles differ substantially. On an NVIDIA A100 GPU, XLM-

Table 3: Macro-F1 across 11 languages on FLORES+, CommonLID, and WiLI-2018 ($N = 7,047$ target + background test sets) under Zero-Shot, Target-Only Fine-Tuned, and Continual Multilingual Rehearsal (Aya) conditions.

Foundation Model	Zero-Shot Pretrained			Target-Only Fine-Tuned			Continual Rehearsal (Aya)		
	FLORES+	CommonLID	WiLI-18	FLORES+	CommonLID	WiLI-18	FLORES+	CommonLID	WiLI-18
XLM-RoBERTa Base	0.350	0.391	0.311	0.117	0.107	0.123	0.874	0.838	0.825
fastText LID-176	0.734	0.772	0.762	0.938	0.972	0.982	0.940	0.949	0.979
ConLID	0.764	0.732	0.762	0.955	0.923	0.971	0.987	0.946	0.969
GlottLID v3	0.762	0.766	0.754	0.990	0.962	0.973	0.995	0.969	0.978
NLLB LID-218	0.766	0.766	0.762	0.997	0.959	0.983	0.995	0.970	0.978
OpenLID-v2	0.583	0.611	0.566	0.917	0.787	0.798	0.851	0.776	0.802

RoBERTa Base requires ≈ 18.52 ms/sentence and > 1.2 GB VRAM. In contrast, adapted fastText LID-176 and GlottLID v3 require only **0.12 ms/sentence on commodity CPU** ($154\times$ faster) with a compact 126 MB memory footprint. For production web crawlers and digital library indexing, subword foundation models deliver both superior Macro-F1 (0.9954 vs. 0.8740) and dramatically lower deployment costs.

5 Discussion and Conclusion

Our findings offer clear guidance for practical NLP deployment:

For Closed Historical Collections & Monastic Archives. When processing pre-filtered historical collections known to reside in Sinhala script, heavy transformer models are unnecessary. Lightweight subword models (MNB or fastText trained from scratch) reach ≥ 0.995 Macro-F1 on full sentences and prove far more dependable on isolated catalog headwords (0.858 F1) without requiring GPU hardware.

For Web Crawlers & Open-World Ingest. When ingesting unfiltered global web text, classical models struggle to scale due to feature vocabulary explosion. Here, practitioners should deploy adapted foundation models like GlottLID or fastText. Crucially, models must not be fine-tuned solely on target data; continual adaptation with a balanced multilingual replay buffer (such as Aya) preserves global language accuracy while resolving target-script ambiguities.

Conclusion. Treating scripts as exclusive proxies for single modern languages has left canonical Pali and classical Sanskrit literature completely unrecognized in modern language technologies (0.0% zero-shot recall). In this work, we created the first dedicated benchmark for these languages in the Sinhala script across closed-world and open-world settings. Our results demonstrate that character

n -gram models best preserve subword morphological distinctions in short fragments, while continual rehearsal with multilingual data eliminates catastrophic forgetting in foundation models, ensuring reliable preservation of historical and liturgical texts within modern language processing pipelines.

Limitations

We note three primary limitations: (1) Approximately 36.8% of our Sanskrit target corpus (4,684 sentences from DCS) was transliterated into Sinhala script using *Aksharamukha*; synthetic transliteration may not reflect all idiosyncratic scribal ligatures found in physical palm leaves. (2) Our benchmarks evaluate clean digital text; severely degraded historical palm leaves with OCR noise require specialized vision preprocessing. (3) While our 11-language hybrid suite samples major Indo-Aryan, Dravidian, and global families, testing across all 200+ world languages remains future work.

Ethics Statement

This research respects the cultural and religious sanctity of Theravada canonical texts (*Tipitaka*) and classical Sanskrit treatises. All data sources used in this work (SiPaKosa, SiDiaC-v2.0, SansinNT, DCS, FLORES+, CommonLID, WiLI-2018, Aya) are public-domain or shared under open academic licenses. Our models and benchmarks are dedicated to cultural heritage preservation and improving representation for low-resource languages in global language technology.

References

- E. W. Adikaram. 1946. *The Early History of Buddhism in Ceylon*. D. S. Puswella, Migoda, Ceylon.
- Laurie Burchell, Alexandra Birch, Nikolay Bogoychev, and Kenneth Heafield. 2023. [An open dataset and model for language identification](#). In *Proceedings of the 61st Annual Meeting of the Association for*

337	<i>Computational Linguistics (Volume 2: Short Papers)</i> , pages 865–879, Toronto, Canada. Association for Computational Linguistics.	394
338		395
339		396
340	Isaac Caswell, Theresa Breiner, Daan van Esch, and Ankur Bapna. 2020. Language ID in the wild: Unexpected challenges on the path to a thousand-language web text corpus . In <i>Proceedings of the 28th International Conference on Computational Linguistics</i> , pages 6588–6608, Barcelona, Spain (Online). International Committee on Computational Linguistics.	397
341		398
342		399
343		400
344		
345		401
346		402
347	Tianqi Chen and Carlos Guestrin. 2016. Xgboost: A scalable tree boosting system. In <i>Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining</i> , pages 785–794.	403
348		404
349		405
350		406
351		
352	Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. 2020. Unsupervised cross-lingual representation learning at scale . In <i>Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics</i> , pages 8440–8451, Online. Association for Computational Linguistics.	407
353		408
354		409
355		410
356		411
357		
358		412
359		413
360		414
361	Marta R Costa-Jussà, James Cross, Onur Çelebi, Maha Elbayad, Kenneth Heafield, Kevin Heffernan, Elahe Kalbassi, Janice Lam, Daniel Licht, Jean Maillard, et al. 2022. No language left behind: Scaling human-centered machine translation. <i>arXiv preprint arXiv:2207.04672</i> .	415
362		416
363		417
364		
365		418
366		419
367	Negar Foroutan, Jakhongir Saydaliev, Grace Kim, and Antoine Bosselut. 2026. ConLID: Supervised contrastive learning for low-resource language identification . In <i>Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)</i> , pages 6693–6708, Rabat, Morocco. Association for Computational Linguistics.	420
368		
369		421
370		422
371		
372		423
373		424
374		425
375	Naman Goyal, Cynthia Gao, Vishrav Chaudhary, Peng-Jen Chen, Guillaume Wenzek, Da Ju, Sanjana Krishnan, Marc’Aurelio Ranzato, Francisco Guzmán, and Angela Fan. 2022. The Flores-101 evaluation benchmark for low-resource and multilingual machine translation . <i>Transactions of the Association for Computational Linguistics</i> , 10:522–538.	426
376		
377		427
378		428
379		429
380		430
381		431
382	Oliver Hellwig. 2010. DCS - the digital corpus of sanskrit. In <i>Proceedings of the 17th World Sanskrit Conference</i> . https://github.com/ambuda-org/dcs .	432
383		433
384		434
385	Tommi Jauregi, Marco Lui, Marcos Zampieri, Timothy Baldwin, and Krister Lindén. 2019. Automatic language identification in texts: A survey . <i>Journal of Artificial Intelligence Research</i> , 65.	435
386		436
387		437
388		438
389	Nevidu Jayatilleke and Nisansa de Silva. 2025. SiDiaC: Sinhala diachronic corpus . In <i>Proceedings of the 39th Pacific Asia Conference on Language, Information and Computation</i> , pages 511–527, Hanoi, Vietnam. Association for Computational Linguistics.	439
390		440
391		441
392		442
393		443
		444
	Armand Joulin, Edouard Grave, Piotr Bojanowski, and Tomas Mikolov. 2017. Bag of tricks for efficient text classification . In <i>Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers</i> , pages 427–431, Valencia, Spain. Association for Computational Linguistics.	445
		446
		447
		448
	Amir Hossein Kargaran, Ayyoob Imani, François Yvon, and Hinrich Schuetze. 2023. GlotLID: Language identification for low-resource languages . In <i>Findings of the Association for Computational Linguistics: EMNLP 2023</i> , pages 6155–6218, Singapore. Association for Computational Linguistics.	
	Marco Lui and Timothy Baldwin. 2012. langid.py: An off-the-shelf language identification tool . In <i>Proceedings of the ACL 2012 System Demonstrations</i> , pages 25–30, Jeju Island, Korea. Association for Computational Linguistics.	
	Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. 2011. Scikit-learn: Machine learning in python. <i>the Journal of machine Learning research</i> , 12:2825–2830.	
	Addison Phillips and Mark Davis. 2009. Tags for identifying languages . RFC 5646, Internet Engineering Task Force.	
	Vinodh Rajan. 2023. Aksharamukha: Asian script converter. https://aksharamukha.appspot.com/ .	
	Shivalika Singh, Freddie Vargus, Daniel D’souza, et al. 2024. Aya dataset: An open-access collection for multilingual instruction tuning. <i>arXiv preprint arXiv:2402.06619</i> .	
	Pedro Ortiz Suarez, Laurie Burchell, Catherine Arnett, Rafael Mosquera, Sara Hincapié Monsalve, Thom Vaughan, Damian Stewart, Malte Ostendorff, Idris Abdulmumin, Vukosi Marivate, Shamsuddeen Hassan Muhammad, Atnafu Lambebo Tonja, Hend Al-Khalifa, Nadia Ghezaïel Hammouda, Verrah Akinyi Otiende, Tack Hwa Wong, Jakhongir Saydaliev, Melika Nobakhtian, Muhammad Ravi Shulthan Habibi, and 78 others. 2026. CommonLID: Re-evaluating state-of-the-art language identification performance on web data . In <i>Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)</i> , pages 33063–33080, San Diego, California, United States. Association for Computational Linguistics.	
	Martin Thoma. 2018. The wili benchmark dataset for written language identification. <i>arXiv preprint arXiv:1801.07779</i> .	
	Maurice Walshe. 1983. Pali literature, including the canonical literature in prakrit and sanskrit of all the hinayana schools . k. r. norman. <i>Buddhist Studies Review</i> , 1(2):172–173.	

Xiang Zhang, Junbo Zhao, and Yann LeCun. 2015. Character-level convolutional networks for text classification. *Advances in neural information processing systems*, 28.

A Detailed Hybrid Benchmark Results Across 11 Languages

Tables 4, 5, and 6 report the comprehensive per-class F1 evaluation across all 11 target and background languages on FLORES+, CommonLID, and WiLI-2018 under Zero-Shot, Target-Only Fine-Tuned, and Continual Rehearsal (Aya) conditions.

B Dataset Provenance and Source Breakdown

Table 7 summarizes the provenance, orthographic representation, textual domains, sentence distributions, and repository links for all primary sources assembled into our 74,318-sentence target benchmark.

C Baseline Model Architectures and Hyperparameters

All seven from-scratch baseline models were trained using standardized configurations:

- **Multinomial Naive Bayes:** Scikit-learn (Pedregosa et al., 2011); character n -gram range $[1, 5]$; sublinear term frequency scaling; Laplace smoothing $\alpha = 0.1$.
- **Linear SVM:** Scikit-learn LinearSVC; character n -grams $[1, 5]$; sublinear TF-IDF; squared hinge loss; $C = 1.0$; max iterations 2,000.
- **Logistic Regression:** Scikit-learn LogisticRegression; character n -grams $[1, 5]$; multinomial cross-entropy; L-BFGS solver; $C = 1.0$; max iterations 1,000.
- **XGBoost:** XGBClassifier (Chen and Guestrin, 2016); 100 estimators; learning rate $\eta = 0.1$; max depth 6; subsample ratio 0.8; colsample_bytree 0.8; multi:softprob objective.
- **Char-CNN:** PyTorch implementation (Zhang et al., 2015); character vocab size 200; embedding dim 64; 3 parallel 1D convolution banks with kernel sizes 3, 5, 7 (128 filters each); ReLU; global max-pooling; dropout $p = 0.3$; AdamW (lr 10^{-3} , weight decay 10^{-4}); batch size 64; 10 epochs with early stopping.

- **Char-BiGRU:** PyTorch implementation; character embedding dim 64; 1-layer bidirectional GRU with 128 hidden units per direction; dropout $p = 0.3$; linear head ($256 \rightarrow 3$); AdamW (lr 10^{-3}); batch size 64; 10 epochs.

- **fastText (From Scratch):** C++ binary / Python API (Joulin et al., 2017); character subwords $[3, 6]$; embedding dim 100; lr $\alpha = 0.5$; 10 epochs; hierarchical softmax loss; bucket size 2,000,000.

D Mathematical Analysis of Classical Scalability Bottlenecks

In Section 3, we noted that classical models (Linear SVM and Multinomial Naive Bayes) fail to scale to open-world multi-lingual settings despite high closed-world accuracy. Here, we formalize the four primary mathematical constraints:

1. Feature Vocabulary Expansion $\mathcal{O}(|V|)$. Let Σ_l denote the character alphabet of language l . For a closed script (Sinhala: $l \in \{\text{sin}, \text{pli}, \text{san}\}$), $|\Sigma_{\text{Sinh}}| \approx 80$ Unicode codepoints (U+0D80–U+0DFF), yielding $|V_{\text{target}}| = \sum_{n=1}^5 |\Sigma_{\text{Sinh}}|^n \approx 4.5 \times 10^4$ observed character n -grams. However, across $K \geq 176$ languages spanning over 30 distinct Unicode writing systems (Latin, Cyrillic, Arabic, Devanagari, Hanzi, etc.), the global alphabet expands to $|\Sigma_{\text{global}}| > 2,500$ codepoints. The observed character n -gram feature space explodes to $|V_{\text{global}}| > 2.5 \times 10^7$ dimensions. For kernel machines, computing the dual optimization problem scales as $\mathcal{O}(N^2 \cdot |V|)$, exceeding RAM bounds when training on web-scale datasets.

2. Linear Prediction Complexity $\mathcal{O}(K \cdot D)$. Flat multinomial classifiers compute class probabilities via dot products against K class weight vectors:

$$P(y = k | x) = \frac{\exp(w_k^T \phi(x))}{\sum_{j=1}^K \exp(w_j^T \phi(x))} \quad (1)$$

Evaluating this across all classes incurs an asymptotic latency of $\mathcal{O}(K \cdot D)$ per prediction. For high-throughput web crawlers processing billions of tokens daily, this linear dependence on K creates an unacceptable inference bottleneck. Modern subword models like fastText circumvent this by organizing labels into a binary Huffman tree (Hierarchical Softmax), reducing evaluation cost to $\mathcal{O}(\log_2 K)$ (Joulin et al., 2017).

Table 4: FLORES+ Hybrid Benchmark Performance: Per-class F1 across all 11 target and global background languages.

Model	Sinh-Sinh	Pali-Sinh	San-Sinh	San-Deva	Eng-Latn	Tam-Taml	Hin-Deva	Ben-Beng	Ara-Arab	Fre-Latn	Ger-Latn	Macro-F1
<i>Zero-Shot Foundation Models (Pretrained Baseline)</i>												
XLM-R Base	–	–	–	–	0.9985	–	0.1835	–	0.6671	0.9990	1.0000	0.3498
ConLID	0.5509	–	–	0.9985	0.9960	0.9995	0.9970	1.0000	0.6255	1.0000	0.9970	0.7422
fastText LID-176	0.7912	–	–	0.9594	0.7213	1.0000	0.9629	1.0000	0.6667	0.9873	0.9897	0.7344
GlottLID v3	0.5451	–	–	0.9945	1.0000	1.0000	0.9921	0.9995	0.9243	1.0000	1.0000	0.7687
NLLB LID-218	0.5442	–	–	0.9890	0.9844	1.0000	0.9907	1.0000	0.9946	0.9995	0.9995	0.7729
OpenLID-v3	0.5444	–	–	0.0311	0.9990	1.0000	0.0422	0.9980	0.6667	0.9950	1.0000	0.5831
<i>Target-Only Fine-Tuned (3 Target Languages: Sinhala, Pali, Sanskrit)</i>												
XLM-R Base	0.1875	0.8580	0.9989	–	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.1170
ConLID	0.9656	0.9754	0.9519	0.9985	0.9960	0.9995	0.9970	1.0000	0.6181	1.0000	0.9970	0.9545
fastText LID-176	0.9917	0.9927	0.9839	0.9829	0.7074	1.0000	0.9946	1.0000	0.6667	1.0000	0.9966	0.9379
GlottLID v3	0.9956	0.9957	0.9943	0.9955	1.0000	1.0000	0.9931	0.9995	0.9122	1.0000	1.0000	0.9896
NLLB LID-218	0.9969	0.9965	0.9951	0.9905	1.0000	1.0000	0.9926	1.0000	0.9951	0.9995	0.9995	0.9969
OpenLID-v3	0.8172	0.8773	0.5252	0.0000	0.9995	1.0000	0.9960	1.0000	0.6667	0.9975	0.9995	0.9172
<i>Continual Adaptation with Multilingual Rehearsal (Target + Global Languages)</i>												
XLM-R Base	0.7310	0.9836	0.9992	–	0.9646	0.9773	0.9975	0.9208	0.5239	0.8743	0.9995	0.8740
ConLID	0.9616	0.9726	0.9409	0.9985	0.9990	0.9995	0.9970	1.0000	0.9877	1.0000	0.9975	0.9868
fastText LID-176	0.9766	0.9881	0.9817	0.9885	0.8178	0.9267	1.0000	1.0000	0.6667	0.9956	0.9931	0.9395
GlottLID v3	0.9826	0.9939	0.9936	1.0000	0.9907	1.0000	0.9980	0.9995	0.9911	0.9995	1.0000	0.9954
NLLB LID-218	0.9852	0.9964	0.9947	1.0000	0.9717	1.0000	0.9995	1.0000	0.9951	1.0000	1.0000	0.9948
OpenLID-v3	0.8674	0.8961	0.5862	0.0000	0.8271	1.0000	1.0000	1.0000	0.6667	0.9208	0.9995	0.8508

Table 5: CommonLID Hybrid Benchmark Performance: Per-class F1 across all 11 target and global background languages.

Model	Sinh-Sinh	Pali-Sinh	San-Sinh	San-Deva	Eng-Latn	Tam-Taml	Hin-Deva	Ben-Beng	Ara-Arab	Fre-Latn	Ger-Latn	Macro-F1
<i>Zero-Shot Foundation Models (Pretrained Baseline)</i>												
XLM-R Base	–	–	–	–	0.9473	–	0.4403	–	0.9955	0.9483	0.9677	0.3908
ConLID	0.5509	–	–	0.9624	0.8502	0.9818	0.9447	0.9620	0.7683	0.8984	0.8955	0.7104
fastText LID-176	0.7910	–	–	0.8886	0.9724	0.9643	0.9702	0.9936	0.9947	0.9447	0.9673	0.7715
GlottLID v3	0.5451	–	–	0.9667	0.9261	0.9878	0.9714	0.9789	0.9648	0.9267	0.9218	0.7445
NLLB LID-218	0.5442	–	–	0.9314	0.9244	0.9747	0.9641	0.9863	0.9940	0.9302	0.9415	0.7446
OpenLID-v3	0.4788	–	–	0.0921	0.8928	0.9750	0.1879	0.9722	0.9945	0.8893	0.9222	0.6106
<i>Target-Only Fine-Tuned (3 Target Languages: Sinhala, Pali, Sanskrit)</i>												
XLM-R Base	0.0389	0.8141	0.9989	–	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.1074
ConLID	0.9656	0.9746	0.9497	0.9624	0.8506	0.9818	0.9447	0.9620	0.7629	0.8986	0.8959	0.9226
fastText LID-176	0.9901	0.9924	0.9705	0.9259	0.9335	0.9938	0.9665	0.9933	0.9899	0.9629	0.9748	0.9721
GlottLID v3	0.9759	0.9890	0.9921	0.9645	0.9294	0.9818	0.9618	0.9805	0.9610	0.9272	0.9214	0.9622
NLLB LID-218	0.9690	0.9612	0.9929	0.9329	0.9033	0.9875	0.9574	0.9866	0.9936	0.9181	0.9420	0.9586
OpenLID-v3	0.7440	0.5081	0.4720	0.0000	0.8615	0.9756	0.9592	0.9839	0.9912	0.8874	0.8961	0.7865
<i>Continual Adaptation with Multilingual Rehearsal (Target + Global Languages)</i>												
XLM-R Base	0.8243	0.9707	0.9992	–	0.9606	0.9364	0.9813	0.9611	0.9958	0.9472	0.9684	0.8381
ConLID	0.9616	0.9725	0.9386	0.9609	0.8882	0.9818	0.9491	0.9637	0.9884	0.9042	0.8984	0.9461
fastText LID-176	0.9766	0.9881	0.9695	0.9303	0.9346	0.7297	0.9752	0.9953	0.9936	0.9638	0.9775	0.9486
GlottLID v3	0.9826	0.9939	0.9913	0.9481	0.9616	0.9878	0.9712	0.9822	0.9940	0.9371	0.9132	0.9694
NLLB LID-218	0.9852	0.9911	0.9925	0.9309	0.9497	0.9875	0.9605	0.9855	0.9939	0.9392	0.9509	0.9697
OpenLID-v3	0.8133	0.7038	0.5476	0.0000	0.9319	0.9756	0.9794	0.9877	0.9961	0.9183	0.9162	0.7763

Table 6: WiLI-2018 Hybrid Benchmark Performance: Per-class F1 across target and background languages.

Model	Sinh-Sinh	Pali-Sinh	San-Sinh	San-Deva	Eng-Latn	Tam-Taml	Hin-Deva	Ben-Beng	Ara-Arab	Fre-Latn	Ger-Latn	Macro-F1
<i>Zero-Shot Foundation Models (Pretrained Baseline)</i>												
XLM-R Base	–	–	–	–	0.9527	–	0.1811	–	–	0.9900	0.9899	0.3114
ConLID	0.5509	–	–	0.9985	0.9448	0.9944	0.9904	0.9435	–	0.9818	0.9707	0.7375
fastText LID-176	0.7912	–	–	0.9904	0.9385	0.9950	0.9889	0.9529	–	0.9774	0.9854	0.7620
GlottLID v3	0.5451	–	–	0.9975	0.9386	0.9949	0.9184	0.9451	–	0.9879	0.9754	0.7303
NLLB LID-218	0.5442	–	–	0.9969	0.9310	0.9939	0.9894	0.9429	–	0.9899	0.9899	0.7378
OpenLID-v3	0.7912	–	–	0.0276	0.9427	0.9940	0.0134	0.9424	–	0.9687	0.9817	0.5662
<i>Target-Only Fine-Tuned (3 Target Languages: Sinhala, Pali, Sanskrit)</i>												
XLM-R Base	0.2209	0.9929	0.9989	–	0.0000	0.0000	0.0000	0.0000	–	0.0000	0.0000	0.1234
ConLID	0.9656	0.9754	0.9471	0.9985	0.9443	0.9944	0.9904	0.9435	–	0.9818	0.9707	0.9712
fastText LID-176	0.9917	0.9927	0.9921	0.9919	0.9381	0.9950	0.9899	0.9507	–	0.9891	0.9904	0.9822
GlottLID v3	0.9956	0.9957	0.9895	0.9975	0.9376	0.9949	0.9107	0.9451	–	0.9869	0.9738	0.9727
NLLB LID-218	0.9969	0.9964	0.9902	0.9969	0.9452	0.9939	0.9894	0.9429	–	0.9899	0.9899	0.9832
OpenLID-v3	0.8172	0.8685	0.5252	0.0000	0.9203	0.9950	0.9843	0.9446	–	0.9742	0.9617	0.7977
<i>Continual Adaptation with Multilingual Rehearsal (Target + Global Languages)</i>												
XLM-R Base	0.8473	0.9949	0.9992	–	0.9012	0.9930	0.9899	0.9733	–	0.9808	0.9698	0.8247
ConLID	0.9616	0.9726	0.9361	0.9990	0.9381	0.9944	0.9904	0.9435	–	0.9844	0.9712	0.9691
fastText LID-176	0.9766	0.9881	0.9895	0.9919	0.9565	0.9749	0.9899	0.9523	–	0.9847	0.9889	0.9793
GlottLID v3	0.9826	0.9939	0.9887	0.9985	0.9225	0.9949	0.9853	0.9446	–	0.9880	0.9759	0.9775
NLLB LID-218	0.9852	0.9964	0.9899	0.9980	0.9068	0.9944	0.9889	0.9424	–	0.9890	0.9904	0.9781
OpenLID-v3	0.8674	0.9151	0.5864	0.0000	0.8695	0.9950	0.9904	0.9446	–	0.9827	0.9663	0.8024

Table 7: Target Corpus Provenance and Source Breakdown for Sinhala, Pali, and Sanskrit in Sinhala Script ($N = 74,318$).

Language Class	Primary Source / Repository	Sentences	Orthography	Genre / Domain	Public Access Repository
Sinhala (Sinh-Sinh)	SiPaKosa-Sent	20,418	Native Sinhala	Buddhist Commentary & History	hf.co/datasets/RaniduG/SiPaKosa-Sent
	Pali-Sinhala Parallel	11,845	Native Sinhala	Canonical Translation Exegesis	hf.co/datasets/sinhala-nlp/pali-sinhala
Pali (Pali-Sinh)	SiDiaC-v.2.0	17,472	Native Sinhala	Canonical Tipitaka & Atthakatha	github.com/NeviduJ/SiDiaC-v.2.0
	Pali-Sinhala Parallel	11,845	Native Sinhala	Canonical Sutta Verses	hf.co/datasets/sinhala-nlp/pali-sinhala
Sanskrit (San-Sinh)	SansinNT	7,953	Native Sinhala	Sanskrit New Testament (27 Books)	ebible.org/details.php?id=sansin
	DCS (Kalidasa, Asvaghosa)	4,684	Transliterated	Classical Kavya & Shastra	github.com/ambuda-org/dcs
	SiDiaC (Kavacha Sangrahaya)	101	Native Sinhala	Protective Mantras & Stotras	github.com/NeviduJ/SiDiaC-v.2.0
Total Target Corpus		74,318			

3. Orthogonal Feature Disconnection. Classical bag-of- n -grams represent features as orthogonal basis vectors: $\langle e_i, e_j \rangle = 0$ for $i \neq j$. Consequently, shared etymological roots across related Indo-Aryan languages (e.g., Sanskrit *dharmā*, Pali *dhamma*, Hindi *dharm*, Sinhala *dahama*) provide zero parameter sharing across language boundaries. In contrast, continuous embedding models map subwords into dense spaces $\mathbb{R}^d$, enabling cross-lingual morphological transfer.

4. Corpus Balancing and Ingestion Constraints. Pretraining classical models from scratch across 200+ languages requires scraping, deduplicating, and balancing petabytes of web text to prevent dominant languages from overwhelming low-resource classes. Adapting existing foundation models that have already absorbed these crawls via continual rehearsal is drastically more data- and compute-efficient.